

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I _____ (Devadharshini E/ 192524483) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
 - Rationality
 - Environment type
- d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

ASSESSMENT OVERVIEW

This document provides complete, step-by-step solutions for all three problems in CSA17 Assessment Tool 2. The solutions emphasize constraint analysis, feasible and optimal solution development, application of AI concepts, and logical justification.

Question	Problem Type	Main AI Method	Final Outcome
1	Doctor-shift scheduling CSP	Backtracking Search	D1-Morning, D2-Afternoon, D3-Night
2	5 x 5 robot path finding	A* (BFS optimality noted)	Optimal path cost = 8
3	Partially observable rescue	Online Uniform Cost Replanning	Safe dynamic replanning

1. HOSPITAL DOCTOR-SHIFT SCHEDULING USING BACKTRACKING

1.1 PROBLEM DEFINITION

Assign three doctors D1, D2, and D3 to Morning, Afternoon, and Night shifts. Each doctor must receive exactly one shift, and each shift must contain exactly one doctor.

1.2 VARIABLES, DOMAINS, AND CONSTRAINTS

Variable	Initial Domain	Reason
D1	{Morning, Afternoon}	D1 cannot work Night
D2	{Morning, Afternoon, Night}	No direct shift restriction
D3	{Afternoon, Night}	D3 cannot work Morning

- C1: $D1 \neq \text{Night}$.
- C2: $\text{shift}(D2) < \text{shift}(D3)$, where Morning < Afternoon < Night.
- C3: All assigned shifts must be different.

- C4: D3 != Morning.
- C5: Every doctor receives exactly one shift.

1.3 BACKTRACKING SEARCH STRATEGY

The search assigns doctors in the order D1 -> D2 -> D3. At each step, a candidate shift is tested against all constraints. If a constraint fails, that value is rejected and the algorithm tries the next value. If no value remains, it backtracks to the previous doctor.

1.4 INTERMEDIATE STEPS AND CONSTRAINT CHECKS

Step	Attempt	Partial Assignment	Constraint Check	Decision
0	Start	{}	No variables assigned	Continue
1	D1 = Morning	{D1=M}	D1 is not Night; no duplicate	Accept
2	D2 = Morning	{D1=M, D2=M}	Two doctors in Morning	Reject; try next D2 value
3	D2 = Afternoon	{D1=M, D2=A}	Different shifts; a later shift exists for D3	Accept
4	D3 = Afternoon	{D1=M, D2=A, D3=A}	Duplicate shift and D2 is not before D3	Reject
5	D3 = Night	{D1=M, D2=A, D3=N}	All constraints satisfied	Goal reached

1.5 SEARCH TREE

```

D1
|
+-- Morning
    |
    +-- D2 = Morning -> Reject (same shift as D1)
    |
    +-- D2 = Afternoon
        |
        +-- D3 = Afternoon -> Reject (duplicate; D2 is not before D3)
        |
        +-- D3 = Night -> VALID SOLUTION
  
```

1.6 FINAL VALID SCHEDULE

Shift	Assigned Doctor
Morning	D1
Afternoon	D2
Night	D3

Final assignment: D1 = Morning, D2 = Afternoon, D3 = Night

1.7 FINAL CONSTRAINT VERIFICATION

Constraint	Verification	Result
D1 cannot work Night	D1 is assigned Morning	Satisfied
D2 must work before D3	Afternoon occurs before Night	Satisfied
Only one doctor per shift	M:D1, A:D2, N:D3	Satisfied
D3 cannot work Morning	D3 is assigned Night	Satisfied
Every doctor assigned once	D1, D2, and D3 each appear once	Satisfied

1.8 BACKTRACKING PSEUDOCODE

```
BACKTRACK(assignment):
    if all doctors are assigned:
        return assignment

    doctor = select next unassigned doctor

    for each shift in DOMAIN(doctor):
        if shift is consistent with all constraints:
            assign doctor = shift

            result = BACKTRACK(assignment)
            if result is not failure:
                return result

    remove doctor from assignment

    return failure
```

1.9 INTERPRETATION

Backtracking avoids enumerating every complete schedule. It detects conflicts as soon as they appear and rejects invalid partial assignments. The first valid schedule found is feasible and satisfies all five constraints.

2. ROBOT PATH FINDING IN A 5 X 5 GRID

2.1 CLARIFICATION AND ASSUMED GRID

Because the assessment sheet does not include the actual grid layout, the following 5 x 5 grid is used for the complete demonstration. Coordinates are written as (row, column). Start S = (1,1), Goal G = (5,5), and # represents an obstacle.

Assumed 5 x 5 Grid

S	.	#	.	.
.	.	#	.	.
#	.	.	.	#
.	#	#	.	.
.	.	.	.	G

Legend: S = Start, G = Goal, # = Obstacle, . = Free cell

2.2 PROBLEM FORMULATION

Component	Definition
Initial state	Robot at S = (1,1)
Goal state	Robot reaches G = (5,5)
Actions	Up, Down, Left, Right
Transition	Move to an adjacent free cell inside the grid
Step cost	1 for every legal movement
Heuristic	$h(n) = \text{row_goal} - \text{row_n} + \text{column_goal} - \text{column_n} $

2.3 WHY THE WORKED ALGORITHM IS A*

BFS uses a FIFO queue and expands nodes by depth. It does not use Manhattan distance. The instruction to use Manhattan distance and show priority-queue updates corresponds to A*, which orders nodes by:

$$f(n) = g(n) + h(n)$$

Here, $g(n)$ is the path cost from S, and $h(n)$ is Manhattan distance to G. Because every move costs 1, ordinary BFS also guarantees a shortest path. A* normally expands fewer nodes by using the heuristic.

2.4 HEURISTIC VALUES

For any cell (r,c) : $h(r,c) = |5-r| + |5-c|$.

- At S(1,1): $h = |5-1| + |5-1| = 8$.
- At G(5,5): $h = 0$.
- Manhattan distance never overestimates the remaining cost for four-directional movement, so it is admissible.

2.5 NODE EXPANSION AND PRIORITY-QUEUE UPDATES

Tie-breaking rule: lowest f first, then lowest h ; when values are equal, newly generated moves are considered in the order Right, Down, Left, Up.

Step	Expanded Node	g	h	f	Priority Queue after Expansion (lowest first)
0	-	-	-	-	[(1,1): g0 h8 f8]
1	(1,1) S	0	8	8	[(1,2):1,7,8; (2,1):1,7,8]
2	(1,2)	1	7	8	[(2,2):2,6,8; (2,1):1,7,8]
3	(2,2)	2	6	8	[(3,2):3,5,8; (2,1):1,7,8]
4	(3,2)	3	5	8	[(3,3):4,4,8; (2,1):1,7,8]
5	(3,3)	4	4	8	[(3,4):5,3,8; (2,1):1,7,8]
6	(3,4)	5	3	8	[(4,4):6,2,8; (2,1):1,7,8; (2,4):6,4,10]
7	(4,4)	6	2	8	[(4,5):7,1,8; (5,4):7,1,8; (2,1):1,7,8; (2,4):6,4,10]
8	(4,5)	7	1	8	[(5,5):8,0,8; (5,4):7,1,8; (2,1):1,7,8; (2,4):6,4,10]

Step 9: remove (5,5) from the priority queue. It is the goal, so the search stops. The parent links are followed backward to reconstruct the optimal path.

2.6 OPTIMAL PATH AND COST

S(1,1) -> (1,2) -> (2,2) -> (3,2) -> (3,3)
-> (3,4) -> (4,4) -> (4,5) -> G(5,5)

Number of moves = 8 Total path cost = 8

2.7 OPTIMALITY PROOF

- The Manhattan distance from S to G is 8.
- Any legal route must contain at least 8 moves because each move changes one coordinate by only 1.
- The discovered path has exactly 8 moves.
- Therefore, no route can have cost less than 8, and the path is optimal.

2.8 A* PSEUDOCODE

```
A_STAR(start, goal):
  OPEN = priority queue ordered by  $f = g + h$ 
  insert start with  $g(\text{start}) = 0$ 
  PARENT[start] = null

  while OPEN is not empty:
    current = remove node with smallest  $f$ 

    if current = goal:
      return reconstruct_path(PARENT),  $g(\text{current})$ 

    for each legal neighbor of current:
      tentative_g =  $g(\text{current}) + 1$ 

      if neighbor is unvisited
        or tentative_g <  $g(\text{neighbor})$ :

        PARENT[neighbor] = current
         $g(\text{neighbor}) = \text{tentative\_g}$ 
         $h(\text{neighbor}) = \text{Manhattan}(\text{neighbor}, \text{goal})$ 
         $f(\text{neighbor}) = g(\text{neighbor}) + h(\text{neighbor})$ 
        insert or update neighbor in OPEN

  return failure
```

2.9 BFS EQUIVALENCE FOR THIS PROBLEM

If the teacher requires literal BFS, replace the priority queue with a FIFO queue and ignore h and f . Since every edge has cost 1, BFS will also return a minimum-cost path of 8, although its node-expansion order may include more cells.

3. AUTONOMOUS RESCUE ROBOT IN A PARTIALLY OBSERVABLE BUILDING

3.1 PROBLEM UNDERSTANDING

The rescue robot must reach a survivor while avoiding blocked cells, considering risky zones, and discovering the building gradually. It cannot construct a complete fixed plan at the beginning because it observes only adjacent rooms. The correct solution must combine sensing, internal-state updates, least-cost planning, action, and replanning.

3.2 CONSTRAINT ANALYSIS

Constraint	Effect on Decision Making	Required Response
Four-direction movement	Limits successor states to adjacent rooms	Generate only Up, Down, Left, Right actions
Blocked cells	Some actions are illegal and may invalidate a plan	Mark obstacle cost as infinity and replan
Adjacent-cell sensing	The map is incomplete and changes as the robot moves	Maintain and update an internal world model
Normal move cost = 1	Shorter routes are generally preferred	Accumulate path cost $g(n)$
Risk adds 2	Entering a risky cell costs 3 in total	Prefer safe cells unless the detour costs more
Minimize cost safely	Distance alone is insufficient	Optimize movement cost plus risk penalty

3.3 COST MODEL

Cell Type	Entry Cost	Meaning
Normal free cell	1	Standard movement
Risky cell R	3	Movement cost 1 + risk penalty 2
Obstacle #	Infinity / illegal	Robot cannot enter

3.4 RECOMMENDED AGENT AND SEARCH STRATEGY

Recommended approach: Model-based, utility-based online agent with Uniform Cost Search replanning. Uniform Cost Search is used on the robot's

current known map because action costs are not uniform: normal cells cost 1 and risky cells cost 3. After each movement, new sensor information is added to the map and the least-cost route is recalculated.

3.5 ONLINE SEARCH CYCLE

1. Sense the adjacent cells.
2. Update the internal map: free, risky, blocked, or still unknown.
3. Run Uniform Cost Search from the current cell to G on the known map.
4. If G is not yet reachable on the known map, choose the lowest-cost unexplored frontier.
5. Execute only the first action of the current plan.
6. Sense again; if a new obstacle or risk is found, discard the invalid portion and replan.
7. Stop when G is reached and the survivor is located.

3.6 DEMONSTRATION GRID

The following example illustrates online replanning. The robot initially believes that cell (3,5) is traversable. When it reaches (3,4), adjacent sensing reveals that (3,5) is actually blocked, so it must choose a riskier alternative.

Example Disaster-Building Grid

S	.	.	#	.
#	R	.	#	.
.	R	.	.	#
.	#	#	R	.
.	.	.	.	G

Legend: R = risky cell (entry cost 3), # = blocked, . = normal (entry cost 1)

3.7 DYNAMIC REPLANNING EXAMPLE

Stage	Robot Knowledge / Action	Cost Added	Cumulative Cost
1	S(1,1) -> (1,2)	1	1
2	(1,2) -> (1,3)	1	2

Stage	Robot Knowledge / Action	Cost Added	Cumulative Cost
3	(1,3) -> (2,3) -> (3,3) -> (3,4)	3	5
4	At (3,4), sensor detects (3,5) is blocked	0	5
5	Replan: (3,4) -> risky (4,4)	3	8
6	(4,4) -> (4,5) -> G(5,5)	2	10

Initial planned route (before obstacle discovery):

S -> (1,2) -> (1,3) -> (2,3) -> (3,3)
-> (3,4) -> (3,5) -> (4,5) -> G

New percept at (3,4):
(3,5) is blocked.

Replanned route:
(3,4) -> (4,4)[RISK] -> (4,5) -> G

Final executed route cost:
5 already spent + 3 risky entry + 1 + 1 = 10

3.8 APPLICATION OF AI CONCEPTS

A. Intelligent Agent

The robot is an intelligent agent because it perceives the environment through sensors, maintains an internal state, chooses actions, and affects the environment through actuators.

PEAS Element	Rescue-Robot Description
Performance measure	Reach survivor, minimize total cost, avoid damage, reduce rescue time
Environment	Rooms, corridors, obstacles, risk zones, smoke, debris, survivor
Actuators	Wheels/tracks, steering, robotic arm, alarm and communication unit
Sensors	Camera, LiDAR, proximity, heat, smoke/gas, sound, location and wheel sensors

B. Rationality

A rational action is the one expected to maximize the rescue objective using the current percepts and knowledge. The robot does not always choose the geometrically shortest route. It chooses the route with the lowest expected total cost while maintaining safety. A risky cell is entered only when its penalty is lower than every known safe alternative or when it is necessary to reach the survivor.

C. Environment Type

Property	Classification	Reason
Observability	Partially observable	Only adjacent cells can be sensed
Determinism	Partly stochastic	Debris, sensing uncertainty, or changing hazards may alter outcomes
Episodic/Sequential	Sequential	Every move affects future reachable states and cost
Static/Dynamic	Dynamic or semi-dynamic	Hazards and blocked routes can change during rescue
Discrete/Continuous	Discrete planning model	Rooms and four movement actions form a grid
Single/Multi-agent	Mainly single-agent	The robot plans its own path; humans may be external entities
Known/Unknown	Initially unknown	The map is learned online

3.9 ONLINE UCS PSEUDOCODE

```
ONLINE_RESCUE_AGENT(start, goal):
    current = start
    knowledge_map = all cells UNKNOWN
    total_cost = 0

    while current != goal:
        percepts = SENSE_ADJACENT_CELLS()
        UPDATE(knowledge_map, percepts)

        path = UNIFORM_COST_SEARCH(
            current,
            goal,
            knowledge_map,
            normal_cost = 1,
            risky_cost = 3,
            obstacle_cost = infinity
        )

        if path does not exist:
            frontier =
LOWEST_COST_UNKNOWN_FRONTIER(knowledge_map)
            path = UNIFORM_COST_SEARCH(current, frontier,
knowledge_map)

        next_cell = SECOND_NODE(path) // execute one step only
        MOVE(next_cell)

        total_cost = total_cost + ENTRY_COST(next_cell)
        current = next_cell

    return "Survivor reached", total_cost
```


3.10 WHY THIS APPROACH IS MOST SUITABLE

Reason	Justification
Handles unequal costs	UCS correctly compares normal-cost and risky-cost routes.
Supports optimality	With non-negative costs, UCS returns the least-cost path on the current known map.
Works with partial information	The online agent does not require a complete map before moving.
Adapts to new obstacles	Replanning immediately responds when a route becomes blocked.
Promotes safety	Risk penalties discourage dangerous cells without making rescue impossible.
Uses rational action	Each decision is based on the best available evidence and the rescue objective.

3.11 FINAL INTERPRETATION

The rescue robot must use a continuous perceive-plan-act cycle rather than a single offline path. In the demonstration, it first follows the best known route, discovers a hidden obstacle, updates its model, and replans through one risky cell. The final route cost is 10. More importantly, the method remains valid for any newly discovered obstacle or risk because planning is repeated after each percept.

OVERALL CONCLUSION

Assessment Tool 2 demonstrates three major AI problem-solving ideas. Backtracking solves a finite constraint-satisfaction problem by rejecting inconsistent assignments. A* uses Manhattan distance and a priority queue to obtain an optimal grid path efficiently, while BFS remains optimal when all movement costs are equal. An online model-based rescue agent combines sensing, Uniform Cost Search, dynamic map updates, rationality, and replanning to operate safely in a partially observable disaster environment.

Question	Technique	Key Result	Why Appropriate
1	Backtracking CSP	D1=M, D2=A, D3=N	Checks constraints during assignment
2	A* / BFS optimality	Optimal cost 8	Manhattan heuristic guides expansion
3	Online UCS agent	Dynamic safe replanning	Handles unknown map and unequal costs